

Thư viện và detector (Libraries and Detectors)

PHỤ THUỘC B/07-detection-pipeline (quy trình phát hiện — chương này chỉ ra chỗ nào trong mã hiện thực hoá từng bước); B/08-corner-refinement (tinh chỉnh góc); A/03-pnp-va-uoc-luong-tu-the và A/04-pose-ambiguity-target-phang (PnP và hai nghiệm — solvePnPGeneric là API quan trọng nhất của cả cây).

TÓM TẮT

Chương này là tăng công cụ cho những gì chạy *mỗi khung hình*: detector và bộ giải PnP. Nó không nhằm liệt kê đủ mà nhằm ba việc cụ thể. Thứ nhất, chỉ ra *một* API quan trọng hơn mọi API khác — solvePnPGeneric, vì nó là API duy nhất trả về nhiều nghiệm kèm sai số tái chiếu của từng nghiệm, tức đại lượng cần để đo ambiguity. Thứ hai, chỉ ra ba tệp mã nguồn đáng *đọc* chứ không chỉ dùng, vì đọc chúng là cách kiểm xem mình đã hiểu lý thuyết đúng chưa. Thứ ba, liệt kê các tham số thực sự đáng chỉnh và nói rõ phần còn lại nên để mặc định. Nếu chỉ nhớ một điều: *nếu pipeline của bạn không lấy ra ρ và hiệp phương sai của pose, bạn đang vứt đi hai đại lượng đã được tính sẵn* — và cả cây này dựa vào chúng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **solvePnPGeneric** — hàm OpenCV giải PnP và trả về *mảng* nghiệm cùng *mảng* sai số tái chiếu tương ứng. Khác với solvePnP vốn chỉ trả một nghiệm.
- cờ **SOLVEPNP_*** — tham số chọn thuật toán: ITERATIVE, EPNP, P3P, IPPE, IPPE_SQUARE, SQPNP và vài cái khác. Chúng không tương đương — xem bảng ở mục 3A.
- **objdetect** — module của OpenCV chứa ArUco và ChArUco từ phiên bản 4.7 trở đi; trước đó chúng nằm ở contrib.
- **wrapper ROS** — gói ROS bọc một thư viện gốc để nó nhận và trả dữ liệu qua topic. Tiện, nhưng nó thường *ẩn* các tham số quan trọng và *vứt* các đại lượng cần đoán.

CÂU HỎI MỞ ĐẦU

- Q1. Trong toàn bộ các API của OpenCV liên quan tới marker, có đúng một hàm mà thiếu nó thì bạn không thực hiện được B/10. Hàm nào, và đại lượng nào nó cho mà các hàm khác giấu?
- Q2. Bạn gọi solvePnP với cờ mặc định trên bốn góc của một tag vuông. Nếu hai điều không tối ưu đang xảy ra, và cách sửa mỗi điều.
- Q3. Có ba tệp mã nguồn trong cả hệ sinh thái đáng đọc chứ không chỉ dùng. Chúng là gì, và mỗi tệp kiểm được sự hiểu của bạn về chương nào?
- Q4. Wrapper ROS cho detector marker tiện lợi nhưng nó thường làm mất hai thứ mà cây này coi là bắt buộc. Hai thứ nào?
- Q5. Chương này chốt tại tháng 9/2026 và sẽ hết hạn. Phần nào của nó hết hạn nhanh nhất, và phần nào gần như không hết hạn?

1.1 Đặt vấn đề

Phần A tới C không nhắc tới một dòng code cụ thể nào, và đó là có chủ ý: hình học phối cảnh không đổi phiên bản, còn tên hàm thì đổi. Chương này là chỗ trả nợ, và nó mang theo một hạn sử dụng.

Mục đích của chương không phải là một danh mục. Các thư viện đều có tài liệu tốt hơn bất cứ thứ gì tôi viết ở đây, và chép lại chúng là vô ích. Mục đích là ba việc mà tài liệu chính thức *không* làm.

Việc thứ nhất: chỉ ra cái gì quan trọng. Tài liệu API liệt kê mọi hàm với cùng trọng số. Nhưng với bài toán của cây này, có đúng một hàm mà thiếu nó thì cả B/10 không thực hiện được, và có ba tham số đáng chỉnh trong khi vài chục tham số khác nên để mặc định. Biết điều đó tiết kiệm nhiều thời gian hơn biết thêm mười hàm.

Việc thứ hai: nói cái gì bị giấu. Nhiều đại lượng chẩn đoán quan trọng *đã được tính* bên trong thư viện rồi bị vứt đi ở giao diện. Tỉ số ambiguity là ví dụ rõ nhất: IPPE tính cả hai nghiệm dù bạn có dùng hay không, nhưng solvePnP chỉ trả về một. Biết chỗ nào có đại lượng bị giấu là biết chỗ nào đáng sửa vài dòng.

Việc thứ ba, và nó là việc có giá trị lâu dài nhất: chỉ ra mã nào đáng đọc như một bài kiểm tra. Đọc `apriltag_pose.c` rồi nhận ra mình hiểu vì sao nó tính hai nghiệm — đó là một phép kiểm xem A/04 đã được hiểu hay chỉ được đọc. Mục 4 dành cho ba tệp như vậy.

CẠM BẮY

Chương này chốt tại tháng 9/2026 và là chương hết hạn nhanh nhất của cây.

Tên hàm đổi, module được chuyển vị trí (ArUco đã chuyển từ contrib sang objdetect ở OpenCV 4.7), tham số mặc định đổi giữa các phiên bản, và các thư viện mới xuất hiện.

Phần *không* hết hạn: các nguyên tắc ở mục 2 (đại lượng nào cần lấy ra), lý do đằng sau việc chọn cờ ở mục 3, và giá trị của việc đọc mã ở mục 4. Chúng vẫn đúng khi tên hàm đã đổi.

Rà lại mỗi sáu tháng, và khi rà thì kiểm bằng cách *chạy* chứ không bằng cách đọc tài liệu — mục 3B là một ví dụ về việc chạy thật phát hiện ra gì.

1.2 Đại lượng phải lấy ra khỏi pipeline

Trước khi nói về hàm nào, nói về *cái gì* phải có ở đầu ra. Ba đại lượng, và cả ba thường bị vứt.

Một, tỉ số ambiguity ρ . Không có nó thì B/10 bậc 2 không thực hiện được, và bạn không biết khi nào pose của mình không đáng tin. Nó đã được tính sẵn bởi IPPE.

Hai, hiệp phương sai của pose $\mathcal{I}^{-1}$. Không có nó thì B/11 không có trọng số, B/09 mục 7 không có cổng chi-square, và B/13 mục 8 không có tiêu chí “phép đo đáng tin”. Nó đã được tính sẵn bởi bước tinh chỉnh — nó chính là ma trận của phương trình chuẩn (A/03 mục 4B).

Ba, hiệp phương sai của từng góc. Không có nó thì trọng số ở A/03 mục 6 là trọng số đều. Nó đã được tính sẵn bởi bước tinh chỉnh góc — nghịch đảo ma trận cấu trúc (B/08 mục 2).

Ba đại lượng, ba lần “đã được tính sẵn”. Đây là chủ đề của chương: *phần lớn thông tin chẩn đoán mà cây này cần đã tồn tại bên trong thư viện, và việc của bạn là lấy nó ra chứ không phải tính lại.*

Chi phí: với ρ thì đổi một lời gọi hàm; với hai đại lượng còn lại thì vài chục dòng, vì OpenCV không xuất chúng qua API công khai. Đây là một thiếu sót đáng tiếc của API và nó là lý do chính đáng nhất để tự viết bước tinh chỉnh PnP thay vì dùng solvePnPRefineLM.

1.3 OpenCV: các API và một phép chạy thật

A. Chọn cờ SOLVEPNP_* nào

Bảng 1.1. Các cờ SOLVEPNP_* và khi nào dùng cho bài toán marker.

Cờ	Số điểm	Dùng khi
IPPE_SQUARE	đúng 4, tag vuông	Mặc định cho một tag vuông. Trả hai nghiệm kèm reprojection ($A/04$)
IPPE	≥ 4 , đồng phẳng	Target phẳng không vuông, chẳng hạn ChArUco một mặt phẳng
SQPNP	≥ 3 , tổng quát	Mặc định cho constellation không đồng phẳng; tối ưu toàn cục ở mức đại số [1]
EPNP	≥ 4 , tổng quát	Nhiều điểm, cần $O(n)$ [2]
P3P / AP3P	đúng 4	Hạt nhân RANSAC, không dùng làm bộ giải chính
ITERATIVE	≥ 4	Cần khởi tạo tốt; hữu ích với useExtrinsicGuess

Câu trả lời cho Q2: gọi `solvePnP` với cờ mặc định (`ITERATIVE`) trên bốn góc tag vuông có hai điều không tối ưu. Thứ nhất, nó *không* dùng cấu trúc phẳng nên nó không cho bạn hai nghiệm — sửa bằng cách chuyển sang `solvePnPGeneric` với `IPPE_SQUARE`. Thứ hai, `ITERATIVE` không có khởi tạo tốt nên nó có thể hội tụ về nghiệm sai — và nó không báo. Sửa bằng cách khởi tạo bằng lời giải dạng đóng rồi mới tinh chỉnh, đúng như $A/03$ mục 9 khuyến nghị.

B. Một phép chạy thật, và điều nó phát hiện

Tôi kiểm các khẳng định trên bằng cách chạy chứ không bằng cách đọc tài liệu, trên OpenCV 4.13.0 ngày 2026-09-14. Hai điều đáng ghi lại.

Điều thứ nhất, xác nhận: `solvePnPGeneric` với `IPPE_SQUARE` trả về đúng hai nghiệm ở mọi tư thế đã thử, kể cả fronto-parallel, kèm hai giá trị sai số tái chiếu. Tỉ số của chúng là ρ , và $C/16$ mục 3B báo cáo nó biến thiên từ 1,33 tới 66,6 theo góc nghiêng. Khẳng định trung tâm của chương — rằng API này cho bạn đại lượng cần thiết — **đã được kiểm.**

Điều thứ hai, một cái bẫy: với tag ở tư thế fronto-parallel *chính xác*, hai nghiệm có sai số tái chiếu bằng nhau tới sai số số học, nên ρ tính được là ∞ hoặc nan tùy vào việc mẫu số có đúng bằng không hay không. Trong dữ liệu không nhiều, tôi gặp cả hai. Mã của bạn *phải* xử lý trường hợp này — kiểm mẫu số trước khi chia — vì nếu không thì ở đúng tư thế nguy hiểm nhất, chỉ báo an toàn của bạn trả về nan và mọi so sánh với ngưỡng đều thành false.

Đây là một ví dụ nhỏ nhưng đáng nhớ của việc vì sao phải chạy: đọc tài liệu không cho bạn biết điều này, và nó là chế độ hỏng ở đúng chỗ mà chỉ báo cần hoạt động nhất.

C. ArUco và ChArUco

Từ OpenCV 4.7, chúng ở `objdetect` thay vì `contrib`. Lớp chính là `ArucoDetector` và `CharucoDetector`.

Tham số đáng chỉnh, và nó khớp với ba tham số ở $B/07$ mục 5C:

`cornerRefinementMethod` — **bật nó.** Mặc định ở một số phiên bản là `CORNER_REFINE_NONE`, và khi ấy bạn đang lấy góc của đa giác xấp xỉ, kém hơn hẳn ($B/08$). Đặt `CORNER_REFINE_SUBPIX` cho ArUco, và với ChArUco thì bước saddle point đã có sẵn trong `interpolateCornersCharuco`.

`minMarkerPerimeterRate` — ngưỡng kích thước tối thiểu, đặt theo con số đo được của A/02 mục 4 chứ không theo mặc định.

`errorCorrectionRate` — số lỗi bit cho phép sửa. Đặt *thấp* và bù bằng whitelist ID (B/14 mục 7A).

Về từ điển: OpenCV có sẵn nhiều từ điển chuẩn, nhưng với docking thì B/07 mục 5C khuyến nghị từ điển *nhỏ, ít ô bit*. `generateCustomDictionary` tạo được từ điển theo yêu cầu, và nó là một trong số ít chỗ mà đi chệch khỏi mặc định gần như luôn đúng.

1.4 Ba tệp mã đáng đọc

Mục này trả lời Q3, và nó là mục có giá trị lâu dài nhất của chương.

Tệp thứ nhất: `apriltag_pose.c` của [apriltag](#). Vài trăm dòng, và nó là toàn bộ A/04 cộng B/10 bậc 2 thành code. Bạn sẽ thấy: IPPE được cài ra sao, hai nghiệm được tính thế nào, và sai số tái chiếu của từng nghiệm được dùng để chọn ra sao. Đọc nó và nhận ra mình *dự đoán được* nó làm gì tiếp là một phép kiểm tốt xem chương 4 đã được hiểu hay chỉ được đọc.

Tệp thứ hai: phần `quad fitting` của [apriltag](#) (thường ở `apriltag_quad_thresh.c`). Đây là B/07 mục 2 thành code, và chỗ đáng chú ý nhất là bước khớp đường có trọng số gradient — cải tiến chính của AprilTag 2 (B/07 mục 2). Đọc nó để thấy vì sao độ chính xác dưới pixel là chuyện có thể.

Tệp thứ ba: phần IBVS của [ViSP](#). Đây là B/13 mục 2 thành code: ma trận tương tác được dựng thế nào, và luật $\mathbf{v} = -\lambda \mathbf{L}^+ \mathbf{e}$ được cài ra sao. Đáng đọc nếu bạn cân nhắc IBVS cho pha cuối.

(Ghi chú trung thực: mô tả về nội dung ba tệp này là từ trí nhớ về cấu trúc của chúng; tôi chưa mở lại mã trong lần soạn này. Tên tệp cụ thể có thể đã đổi. Điều tôi tin chắc là chức năng tồn tại trong các thư viện ấy, không phải tên tệp chính xác.)

Ngược lại, ba thứ đáng dùng mà không cần đọc: [Kalibr](#), [COLMAP](#), [Ceres](#). Chúng là công cụ trưởng thành, tài liệu tốt, và việc đọc mã của chúng không trả lại tương xứng với thời gian bỏ ra.

1.5 Các họ tag khác và detector học sâu

Ngắn, vì B/07 mục 4 đã bàn về mặt kỹ thuật; mục này chỉ nói về mặt công cụ.

[STag](#) [3], [TopoTag](#) [4], [ChromaTag](#) [5]: các họ tag với thiết kế khác. Mã nguồn có, chất lượng khác nhau. Đáng thử nếu bạn có một yêu cầu cụ thể mà AprilTag và ArUco không đáp ứng — chẳng hạn STag khi cần ổn định pose tốt hơn ở tư thế xấu.

[CCTag](#) [6], [WhyCon](#) [7]: marker tròn. Nhớ cảnh báo về bias tâm ellipse (A/02 mục 6) — nếu dùng, phải kiểm xem cài đặt có hiệu chỉnh conic không, và nếu không thì bạn có một dòng hệ thống phụ thuộc tư thế trong ngân sách.

[DeepTag](#) [8], [DeepArUco](#) [9]: detector học sâu. Đáng cân nhắc khi điều kiện quan sát khắc nghiệt và không cải thiện được bằng chiếu sáng (B/14). Với docking — nơi điều kiện kiểm soát được — lợi ích ít rõ, và B/07 mục 4 nêu lý do thận trọng: bền hơn không có nghĩa là chính xác hơn về định vị góc.

[apriltag-generation](#): tạo họ tag riêng với số ô và khoảng cách Hamming tùy chọn. Công cụ để thực hiện đánh đổi ở B/07 mục 5C, và nó đáng dùng hơn vẻ ngoài vì lựa chọn mặc định (dùng họ phổ biến nhất) thường không tối ưu cho docking.

1.6 Wrapper ROS

Mục này trả lời Q4, và nó là một cảnh báo.

`apriltag_ros`, `aruco_ros`, `fiducials`: các gói ROS bọc thư viện gốc. Chúng tiện — nhận ảnh qua topic, trả pose qua TF — và với một hệ thống ROS thì chúng là điểm xuất phát hợp lý.

Nhưng chúng thường làm mất hai thứ mà cây này coi là bắt buộc.

Thứ nhất, ρ . Giao diện ROS thường trả về một pose duy nhất, nên tỉ số ambiguity bị vớt ở tầng wrapper. Hệ quả: bạn không thực hiện được B/10 bậc 2, và bạn không biết khi nào pose không đáng tin.

Thứ hai, hiệp phương sai trung thực. Một số wrapper trả về một trường hiệp phương sai, và nó thường là một hằng số cầu hình chữ không phải $\mathcal{I}^{-1}$ tính từ dữ liệu. Một hiệp phương sai hằng số tệ hơn không có hiệp phương sai, vì nó trông như thông tin thật và mọi tầng trên sẽ tin nó.

Hai mất mát ấy là lý do tôi khuyên: *dùng wrapper để bắt đầu, nhưng sửa nó hoặc thay nó trước khi nghiêm túc.* Việc sửa không lớn — lấy ρ ra và thêm nó vào message — và nó là điều kiện để phần lớn Phần B áp dụng được.

Một điều đáng nêu về `tagslam` [10]: nó khác các wrapper trên ở chỗ nó giải bài toán hợp nhất trên nhiều tag bằng factor graph, tức nó làm đúng việc mà B/09 mục 2 khuyến nghị. Nếu bạn cần map nhiều tag trong môi trường thì nó là điểm xuất phát tốt. Mức tin C — tiền ẩn, nhưng có mã nguồn công khai và được dùng rộng rãi.

1.7 Giới hạn

Chương này không có benchmark. B/07 mục 6 giải thích: tôi không có dữ liệu so sánh được giữa các detector, và trích số từ các bài báo khác nhau ra khỏi ngữ cảnh thì chúng không so sánh được. Nếu cần xếp hạng cho hệ của bạn, mini project của B/07 là cách đo.

Các khẳng định về mã nguồn là từ trí nhớ. Đã ghi ở mục 4. Tên tệp có thể đã đổi; chức năng thì tôi tin là còn.

Chỉ các khẳng định về OpenCV API đã được chạy kiểm, trên đúng một phiên bản (4.13.0) và một hệ điều hành. Hành vi có thể khác ở phiên bản khác — và đặc biệt, vị trí module `ArUco` đã đổi một lần rồi.

Chương chốt tháng 9/2026. Xem cảnh báo ở mục 1.

1.8 Thực hành

Bốn việc.

Một, chuyển sang solvePnPGeneric và lấy ρ ra — việc có tỉ lệ lợi ích trên công sức cao nhất trong chương, và nó là điều kiện cho B/10.

Hai, xử lý trường hợp ρ là nan hoặc vô hạn — mục 3B; nó xảy ra ở đúng tư thế nguy hiểm nhất.

Ba, bật tính chỉnh góc nếu dùng ArUco, và kiểm giá trị mặc định của phiên bản bạn đang dùng thay vì giả định.

Bốn, nếu dùng wrapper ROS, sửa nó để xuất ρ trước khi xây dựng gì phía trên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● API quan trọng nhất của cả cây	<code>solvePnPGeneric</code> — API duy nhất trả về <i>nhiều nghiệm kèm sai số tái chiếu của từng nghiệm</i> , tức ρ . Thiếu nó thì B/10 bậc 2 không thực hiện được.
●●● Ba đại lượng phải lấy ra	ρ (cho B/10); hiệp phương sai pose $\mathcal{I}^{-1}$ (cho B/11, cổng chi-square, tiêu chí hội tụ); hiệp phương sai từng góc (cho trọng số ở A/03 mục 6). Cả ba đã được tính sẵn — việc của bạn là lấy ra, không phải tính lại.
●●● Chọn cờ nào	IPPE_SQUARE cho một tag vuông (cho hai nghiệm); SQPNP cho constellation không đồng phẳng; EPNP khi nhiều điểm; P3P chỉ làm hạt nhân RANSAC.
●●● Hai vấn đề của <code>solvePnP</code> mặc định	Cờ ITERATIVE không dùng cấu trúc phẳng nên không cho hai nghiệm; và nó không có khởi tạo tốt nên có thể hội tụ về nghiệm sai <i>mà không báo</i> .
●●● Cái bẫy ρ tại fronto-parallel	Ở tư thế fronto-parallel chính xác, hai sai số tái chiếu bằng nhau tới sai số số học nên ρ ra nan hoặc ∞ . Phải kiểm mẫu số trước khi chia — nếu không thì ở đúng tư thế nguy hiểm nhất, chỉ báo an toàn trả nan và mọi so sánh ngưỡng thành false. Phát hiện bằng cách CHẠY, không bằng đọc tài liệu.
●● Ba tham số ArUco đáng chỉnh	<code>cornerRefinementMethod</code> (bật — mặc định có thể là NONE); <code>minMarkerPerimeterRate</code> (theo ngưỡng đo được của A/02 mục 4); <code>errorCorrectionRate</code> (thấp, bù bằng whitelist).
●●● Ba tệp mã đáng đọc	<code>apriltag_pose.c</code> (A/04 + B/10 thành code); phần quad fitting của <code>apriltag</code> (B/07 mục 2, khớp đường có trọng số gradient); phần IBVS của <code>ViSP</code> (B/13 mục 2).
●● Ba thứ đáng dùng mà không cần đọc	<code>Kalibr</code> , <code>COLMAP</code> , <code>Ceres</code> — công cụ trưởng thành, tài liệu tốt, đọc mã không trả lại tương xứng.
●●● Hai thứ wrapper ROS làm mất	ρ (giao diện trả một pose duy nhất); và hiệp phương sai <i>trung thực</i> (thường là hằng số cấu hình chứ không phải $\mathcal{I}^{-1}$). Hiệp phương sai hằng số <i>tệ hơn</i> không có, vì nó trông như thông tin thật.
●● Vì sao chương này không có benchmark	Không có dữ liệu so sánh được giữa các detector; trích số từ các bài báo khác nhau ra khỏi ngữ cảnh thì chúng không so được (B/07 mục 6). Cần xếp hạng thì đo trên hệ của mình.
●● Phần nào của chương hết hạn	Tên hàm, vị trí module, giá trị mặc định — hết hạn nhanh. <i>Nguyên tắc</i> (đại lượng nào cần lấy ra, lý do chọn cờ, giá trị của việc đọc mã) — gần như không hết hạn.
● Từ điển tùy chỉnh	<code>generateCustomDictionary</code> — một trong số ít chỗ mà đi lệch khỏi mặc định gần như luôn đúng cho docking, vì bạn chỉ cần vài chục ID (B/07 mục 5C).

●●● phải thuộc ●● nên nắm ● biết là có

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV <code>solvePnPGeneric</code>	Nhiều nghiệm kèm reprojection từng nghiệm	Đã kiểm bằng cách chạy, 4.13.0, 2026-09-14
<code>apriltag</code> (AprilRobotics)	Bản C gốc; <code>apriltag_pose.c</code> và phần quad fitting	Hai trong ba tệp đáng đọc
OpenCV <code>objdetect</code>	ArUco và ChArUco từ 4.7; <code>ArucoDetector</code> , <code>CharucoDetector</code>	Nhớ kiểm mặc định của <code>cornerRefinementMethod</code>
<code>ViSP</code> [11]	IBVS, PBVS, ma trận tương tác, tracker marker	Tệp thứ ba đáng đọc; cho B/13

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
tagslam [10]	Hợp nhất nhiều tag bằng factor graph — đúng việc B/09 mục 2 khuyến nghị	Khác các wrapper ROS khác ở điểm này; mức tin C

MINI PROJECT · Sửa pipeline để xuất ba đại lượng chẩn đoán · 1 ngày

Mục tiêu	biến pipeline của bạn từ một hộp đen trả về một pose thành một hệ đo lường trả về pose kèm bằng chứng về độ tin cậy của nó.
Dữ liệu	chính pipeline hiện tại của bạn.
Các bước	chuyển lời gọi PnP sang solvePnPGeneric với cờ đúng theo bảng ở mục 3A; lấy mảng reprojection ra và tính ρ , có xử lý trường hợp mẫu số bằng không; tự viết bước tinh chỉnh LM (hoặc sửa solvePnPRefineLM) để lấy ra ma trận $\mathcal{I} = \sum J^T \Sigma^{-1} J$ và nghịch đảo nó; nếu dùng ArUco, lấy ma trận cấu trúc từ bước cornerSubPix để có hiệp phương sai từng góc; cuối cùng đối giao diện trả về từ một pose thành cấu trúc ba giá trị của B/10 mục 3C.
Xong khi	pipeline trả về pose, hiệp phương sai, ρ , và một trạng thái ba giá trị; và log của bạn ghi cả ba ở mỗi khung. Kiểm tra chéo bắt buộc: chạy trên một chuỗi có đoạn fronto-parallel và xác nhận rằng ρ không trả về nan ở đó — nếu có thì bạn chưa xử lý trường hợp ở mục 3B, và bạn vừa mất chỉ báo ở đúng chỗ cần nó nhất.
Nối với	B/10 và B/13 mục 8 tiêu thụ trực tiếp ba đại lượng này.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. solvePnPGeneric. Nó là API duy nhất trả về *mảng* nghiệm cùng *mảng* sai số tái chiếu tương ứng; mọi hàm khác — kể cả solvePnP với cùng cờ IPPE — chỉ trả về một nghiệm và vứt phần còn lại. Đại lượng nó cho mà các hàm khác giấu là tỉ số $\rho = e_2/e_1$, tức chỉ báo duy nhất cho biết hai nghiệm có phân biệt được hay không. Thiếu nó thì bậc 2 của B/10 — đo và từ chối khi không tin được — không thực hiện được, và hệ của bạn sẽ chọn nghiệm theo nhiều ở những tư thế mà hai nghiệm ngang nhau, không có cảnh báo nào. Điều đáng nói là IPPE *đã tính* cả hai nghiệm dù bạn có dùng hay không, nên chi phí để lấy ρ ra bằng không — chỉ là đổi một lời gọi hàm.
- A2. Thứ nhất, cờ mặc định ITERATIVE không khai thác cấu trúc phẳng của tag vuông, nên nó không cho bạn hai nghiệm và bạn mất ρ . Sửa: dùng solvePnPGeneric với SOLVEPNP_IPPE_SQUARE. Thứ hai, ITERATIVE là một bộ giải lặp cần điểm khởi đầu, và nếu không truyền useExtrinsicGuess thì nó tự khởi tạo — nên nó có thể hội tụ về nghiệm sai trong hai nghiệm của target phẳng, và nó *không báo* là đã làm thế. Sửa: khởi tạo bằng một lời giải dạng đóng (IPPE cho target phẳng) rồi mới tinh chỉnh, đúng như A/03 mục 9 khuyến nghị. Cả hai lỗi này đều thầm lặng — hàm trả về một pose trông hợp lý trong cả hai trường hợp.
- A3. Một: apriltag_pose.c của [apriltag](#) — nó là A/04 (hai nghiệm của target phẳng) cộng B/10 bậc 2 (dùng tỉ số reprojection để chọn) gói trong vài trăm dòng; đọc nó và thấy mình *dự đoán được* bước tiếp theo là phép kiểm xem chương 4 đã được hiểu hay chỉ được đọc. Hai: phần quad fitting của [apriltag](#) — nó là B/07 mục 2, và chỗ đáng chú ý nhất là bước khớp đường có trọng số gradient, tức cơ chế làm cho độ chính xác dưới pixel khả thi. Ba: phần IBVS của [ViSP](#) — nó là B/13 mục 2, cho thấy ma trận tương tác được dựng thế nào và luật điều khiển được cài ra sao. Ngược lại, [Kalibr](#), [COLMAP](#) và [Ceres](#) thì nên dùng mà không cần đọc: chúng trưởng thành, tài liệu tốt, và đọc mã không trả lại tương xứng thời gian.
- A4. Thứ nhất, tỉ số ambiguity ρ : giao diện ROS thường trả về một pose duy nhất qua TF hoặc một message pose, nên ρ bị vứt ở tầng wrapper — và không có nó thì bạn không biết khi nào pose của mình không đáng tin. Thứ hai, hiệp phương sai *trung thực*: một số wrapper có trường hiệp phương sai trong message, nhưng nó thường được điền bằng một hằng số lấy từ tệp cấu hình chứ không phải $\mathcal{I}^{-1}$ tính từ dữ liệu của khung hiện tại. Mất mát thứ hai tệ hơn mất mát thứ nhất, vì một hiệp phương sai hằng số *tệ hơn không có hiệp phương sai*: nó trông như thông tin thật, nên mọi tầng phía trên — bộ lọc, cổng chi-square, tiêu chí hội tụ — sẽ tin nó và cân các

phép đo sai. Khuyến nghị: dùng wrapper để bắt đầu, nhưng sửa nó để xuất ρ trước khi xây dựng gì phía trên; việc sửa nhỏ và nó là điều kiện để phần lớn Phần B áp dụng được.

- A5. Hết hạn nhanh nhất: tên hàm, vị trí module (ArUco đã chuyển từ contrib sang objdetect ở 4.7 — một lần đổi đã xảy ra rồi), giá trị mặc định của tham số, và danh sách các thư viện mới. Gần như không hết hạn: các *nguyên tắc* — rằng có ba đại lượng chẩn đoán cần lấy ra và cả ba đã được tính sẵn; rằng phải chọn cờ theo cấu trúc hình học của target chứ không theo mặc định; rằng đọc mã là một phép kiểm sự hiểu chứ không chỉ là cách học API; và rằng wrapper tiện lợi thường vứt đi thông tin chẩn đoán. Bốn nguyên tắc ấy vẫn đúng khi mọi tên hàm trong chương đã đổi. Cách rà lại đúng: kiểm bằng cách *chạy* chứ không bằng đọc tài liệu — mục 3B là ví dụ về việc chạy thật phát hiện ra một cái bẫy mà tài liệu không nói.

NÓI LẠI THẬT ĐƠN GIẢN

Phần lớn tài liệu này nói về nguyên lý, và nguyên lý thì không đổi. Chương này nói về tên hàm, và tên hàm thì đổi — nên nó có hạn sử dụng, ghi rõ là tháng 9 năm 2026.

Điều đáng nhớ nhất của chương không phải một tên hàm mà là một quan sát: *phần lớn thông tin bạn cần đã được máy tính sẵn rồi, chỉ là nó bị vứt đi ở cửa ra.*

Ví dụ rõ nhất là chuyện hai câu trả lời của cái tag phẳng. Thư viện *luôn* tính cả hai — nó không có cách nào tính một mà không tính cái kia — nhưng cái hàm thông dụng nhất chỉ trả về một và ném cái còn lại đi. Đổi sang một hàm khác, và bạn có thêm con số cho biết hai câu trả lời có ngang nhau hay không. Đổi một dòng, được một chỉ báo an toàn.

Tương tự với chuyện “máy tin vào kết quả của mình tới mức nào”. Con số đó cũng được tính sẵn trong lúc giải, rồi bị vứt. Lấy nó ra tốn vài chục dòng, và không có nó thì một nửa những việc ở phần trước của tài liệu không làm được.

Có một cái bẫy nhỏ mà tôi chỉ phát hiện được nhờ chạy thử chứ không nhờ đọc tài liệu, và nó đáng kể lại vì nó rất đặc trưng. Cái tỉ số so sánh hai câu trả lời được tính bằng một phép chia. Ở đúng cái tư thế nguy hiểm nhất — tag nhìn thẳng vào camera — hai câu trả lời khớp ảnh ngang nhau hoàn hảo, nên mẫu số bằng không, nên phép chia cho ra một giá trị vô nghĩa. Và một giá trị vô nghĩa thì khi đem so với ngưỡng sẽ luôn ra “không đạt ngưỡng” theo cách im lặng. Nghĩa là: nếu không cẩn thận, cái chỉ báo an toàn của bạn sẽ tắt ngúm đúng vào lúc bạn cần nó nhất.

Cuối cùng, một lời khuyên về việc đọc mã người khác. Có ba tệp đáng đọc trong cả hệ sinh thái này, và lý do đọc chúng không phải để học cách dùng thư viện — tài liệu làm việc đó tốt hơn. Lý do là để *tự kiểm tra mình*: nếu bạn đọc một tệp và thấy mình đoán trước được dòng tiếp theo làm gì, thì bạn đã hiểu cái chương lý thuyết tương ứng. Nếu bạn đọc mà thấy lạ, thì chương ấy mới chỉ được đọc chứ chưa được hiểu.

Chỗ không rút gọn được: tên hàm, tên module và giá trị mặc định sẽ đổi, và không có cách nào viết một chương như thế này cho nó bền. Cách duy nhất là mỗi lần quay lại thì *chạy thử* chứ đừng tin cái đã viết — kể cả cái tôi vừa viết.

Một câu: *Thông tin bạn cần đã được tính sẵn rồi; việc của bạn là đừng để nó bị vứt ở cửa ra.*

1.9 Điều gì chứng minh cách hiểu này sai

Chương công cụ khác chương lý thuyết ở chỗ phần lớn khẳng định của nó kiểm được bằng cách chạy, và một phần đã được kiểm — nên mục này nói về phần còn lại.

Khẳng định trung tâm — solvePnPGeneric với IPPE_SQUARE trả về hai nghiệm kèm sai số tái chiếu, và tỉ số của chúng là một chỉ báo dùng được — **đã được kiểm bằng cách chạy** (mục 3B), trên OpenCV 4.13.0. Nó bị bác bỏ nếu một phiên bản khác trả về một nghiệm, hoặc trả về hai nghiệm mà sai số tái chiếu không phân biệt được chúng ở góc nghiêng lớn. Cách kiểm lại khi nâng cấp: chạy phần 2 của script ở C/16 và xem cột ρ có còn tăng theo góc nghiêng không.

Khẳng định thứ hai — *ba đại lượng chẩn đoán đã được tính sẵn và chỉ cần lấy ra* — bị bác bỏ nếu hoá ra một trong ba không tồn tại ở dạng có thể lấy được. Với ρ thì tôi đã kiểm. Với hiệp phương sai

pose và hiệp phương sai góc thì tôi *chưa* — tôi dẫn ra từ cấu trúc của thuật toán (A/03 mục 4B, B/08 mục 2) chứ không từ việc mở mã OpenCV. Nếu hoá ra chúng bị huỷ trước khi hàm trả về thì lời khuyên “lấy ra” phải đổi thành “tự cài bước tinh chỉnh”, và chi phí của mini project tăng đáng kể.

Khẳng định thứ ba — *ba tệp mã đáng đọc* — là phán đoán chủ quan của tôi về giá trị sư phạm, và nó không bị bác bỏ bằng dữ liệu. Điều *có thể* sai là mô tả về nội dung của chúng, vốn từ trí nhớ và chưa mở lại mã (mục 4).

Cuối cùng, khẳng định về wrapper ROS ở mục 6 dựa trên hiểu biết chung về các gói ấy, *không* phải trên việc đọc mã của chúng trong lần soạn này. Nếu một wrapper cụ thể có xuất ρ thì cảnh báo ở mục 6 không áp dụng cho nó — và đó sẽ là tin tốt đáng ghi lại.

1.10 Kết nối

Chương này là tầng công cụ cho những gì chạy mỗi khung.

Nổi lên trên. B/07-detection-pipeline mô tả quy trình mà mục 3C và 4 chỉ ra chỗ hiện thực hoá. B/08-corner-refinement mô tả các phương pháp mà tham số cornerRefinementMethod chọn giữa. A/03-pnp-va-uoc-luong-tu-the và A/04-pose-ambiguity-target-phang là hai chương mà solvePnPGeneric phục vụ trực tiếp.

Nổi ngang. D/18-cong-cu-calib-servo-va-do-dac là chương anh em: nó bàn những gì chạy *một lần* thay vì mỗi khung, và ranh giới ấy gần trùng với ranh giới “thứ bạn phải tự cài” và “thứ nên dùng đồ có sẵn”.

Nổi xuống. B/10-xu-ly-ambiguity và B/13-docking-control mục 8 tiêu thụ trực tiếp ba đại lượng ở mục 2, nên mini project của chương này là điều kiện tiên quyết cho cả hai.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba đại lượng phải lấy ra khỏi pipeline, chương nào tiêu thụ mỗi cái, và điều chung của cả ba.
2. Vì sao gọi solvePnP với cờ mặc định trên tag vuông là không tối ưu theo hai cách? Nêu cách sửa mỗi cách.
3. Mô tả cái bẫy ρ tại fronto-parallel, và nói vì sao nó nguy hiểm hơn một lỗi thông thường.
4. Ba tệp mã nào đáng đọc, và mỗi tệp kiểm được sự hiểu của bạn về chương nào?
5. Wrapper ROS làm mất hai thứ. Nêu cả hai, và nói vì sao mất mát thứ hai tệ hơn mất mát thứ nhất.
6. Phần nào của chương này hết hạn nhanh nhất và phần nào không? Nêu cách rà lại đúng.
7. Vì sao chương này cố ý không có bảng benchmark so sánh các detector?

Nguồn

- [1] G. Terzakis and M. Lourakis, “A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem,” in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPnP, có trong OpenCV, 2020.
- [2] V. Lepetit, F. Moreno-Noguer, and P. Fua, “EPnP: An Accurate $O(n)$ Solution to the PnP Problem,” *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [3] B. Benligray, C. Topal, and C. Akinlar, “STag: A Stable Fiducial Marker System,” in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.
- [4] G. Yu, Y. Hu, and J. Dai, “TopoTag: A Robust and Scalable Topological Fiducial Marker System,” in *IEEE Transactions on Visualization and Computer Graphics*, mức tin B, 2021.
- [5] J. DeGol, T. Bretl, and D. Hoiem, “ChromaTag: A Colored Marker and Fast Detection Algorithm,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2017.
- [6] L. Calvet, P. Gurdjos, C. Griwodz, et al., “Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [7] T. Krajník, M. Nitsche, J. Faigl, et al., “A Practical Multirobot Localization System,” *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [8] Z. Zhang, Y. Hu, G. Yu, et al., “DeepTag: A General Framework for Fiducial Marker Design and Detection,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin B, 2023.
- [9] R. Berral-Soler, F. J. Madrid-Cuevas, R. Muñoz-Salinas, et al., “DeepArUco: Marker Detection and Classification in Challenging Lighting Conditions,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, mức tin C; tiền ấn/-workshop, dùng làm gợi ý.
- [10] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [11] É. Marchand, F. Spindler, and F. Chaumette, “ViSP for Visual Servoing: A Generic Software Platform with a Wide Class of Robot Control Skills,” *IEEE Robotics & Automation Magazine*, vol. 12, no. 4, pp. 40–52, 2005, mức tin A.